Özyineleme: Bir fonksiyonun kendi kendini çağırması.









Programlama Kavramlarının Tarihi

Spaghetti Code: Mantık akışının bir tabaktaki makarna gibi program boyunca sarıldığı, yapısal olmayan değişsel programlar

Modüler Programming: Programların fonksiyonlarla organize edildiği yapı (hala değişsel kodlama tekniği)

Structured Programming: Bugün hala geçerli:



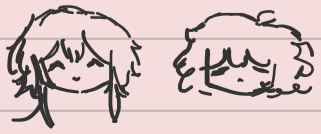
Atomic Data

Atomic data parçalanamaz, bölünemez.

Bir veri yapısı (Data structure), atomik ve kompozit verilerin, tanımlanmış ilişkilerle bir kümede toplanmasıdır.

(anlamlı alt alanlara bölünebilir bütünlük veridir.)








FST

tersten yerdirme kod

hiç bir sorunda bu kadar uğraşmamıştım

data structures
A pseudo code approach to C
Richard Gilberg



```
struct Node{
    int data;
    struct Node* next;
};

void printReverse(struct Node* head){
    if(head == NULL){
        return;
    }
    else{
        printReverse(head -> next);
        printf("%d", head->data);
    }
}
```

liste boş mu diye kontrol edilmeli burda

```
void push(struct Node** head_ref, char new_data){
    // bağlı listenin başını gösteren işaretçinin adresini değiştirmeye çalışıyor.
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node -> data = new_data;
    new_node -> next = (*head_ref);
    (*head_ref) = new_node;

int main(){
    struct Node* head = NULL;
    push(&head, 4);
    push(&head, 3);
    push(&head, 2);
    push(&head, 1);
    printReverse(head);
    return 0;
}
```



Temel Veri Türleri: int, real, char, bool

Veri Yapıları verinin organize bçlı olarak 2 grupta ayrılır. (Data Structure)

- Linear: Array, Linked List, Queue
- Non-linear: Tree, Graph

- Veri yapıları Nested olabilir (iç içe yuvalanmış)

Pseudocode (sözde kod): Gerçekçi ayrıntılar yok. Ne yaptığını söylüyor

ADT (Abstract Data Type): Bir şeyin nasıl çalıştığını bilmiyorsun, sadece ne yapabildiğini bilip kullanıyorsun. "Üstün ne bağını sorma" Kullanıcıdan soyutlayarak için.

* Bir liste ADT'sini implement etmek (gerçekleştirmek) için: Arrays (diziler) ve Linked List (bağlı listeler) kullanabilirsin.

Array Implementations:

- Ardışıl saklanır. Örneğin [10,20,30,40]. Linked listte ise her eleman başka yerde tutup adresleri bağlarız. Elemanların yerleri index (0,1,2,3) ile belirlenir.
- Eleman aramak için çok hızlı ve etkili olsa da silme ve ekleme işleri konusunda ise yavaştır.

Linked List:

- Her bir eleman bir sonraki elemanın konumunu içerir, dolayısıyla bir veri kaybı olmaz.
- Her eleman iki bölümden oluşur: Data (veri) ve bir sonraki data ile bağ (link).

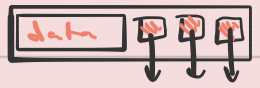
Data kısmı, işlenecek verileri tutar. Listelerse, veriler bir araya getirmek (ilişkilendirmek) için kullanılır. Bunun için pointerlar kullanılır.

* Linear linked listelerde her eleman 0 veya 1 adet pointer'a sahiptir. Linear olmayan bağlı listelerde ise her eleman 0 veya daha fazla sayıda pointer'a sahip olabilir

Node: iki bölümden oluşur



lineer listede bir node



non-lineer

- Data bölümü; tek bir alan, birden çok alan veya birbirine ilişkin bir yapı olabilir. Ancak tek bir alan gibi davranır.
- Diğerlerine göre en büyük avantajı, verilerin kolayca eklenip silinebilmesidir. Yeni bir öğe için yer ayırmak veya bir öğeyi silmek için listedeki öğelerin kaydırılmış gerekmez.
- Ama sıralı aramada (sequential search) yavaştır.

Pointer

Casting: Pointer'ın hangi tür göstereceğini belirtir → int *p;

* int* p;
  float* q;
  ~~p = q;~~ dizeminde farklı uyumsuzlar p = (int*)q; dur.

* void* vp; özel bir göstergedir. Casting olmadan her türlü tutabilir.
  vp = p; ✓
  vp = q; ✓

AMA: Dereferanse edilemez. Yani: void* p;
~~*p = 5;~~ gösterdiği türünü bilmiyorum için.
void, türsüz bir pointerdir. Bunun için önce cast yapmalısın.

void* p = &x;

(int)p = 5;
- (int*) → Casting (türünü belli ediyo)
- * → dereference (gidip içindeki değere bakar)

dereference başka örnek → int x = 10;
int* p = &x;
*p → dereference

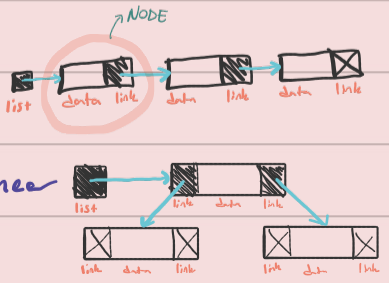
AĞAÇLAR

Infix Traversal → Anne kendini çocuklarının arasına alıyor.
Postfix → kendini sonraya alacak
Prefix → kendini önceye koyacak



ADT (Abstract Data Types)

- **Listeler** → Array (diziler)
  - Linked List (bağlı listeler) → Linear / Non-Linear



- **Stack** → LIFO mantığı ile çalışır
  push(ekle), pop(sıkar), peek(en üsttekine bak)
  pankek tabağına benzer :)
- **Queue** → FIFO
  direk market kuyruğu
- **Tree** → root, parent, child, leaf gibi yapılar
- **Heap** → bir tree türüdür En büyük/küçük elemana hızlı erişim
  Max Heap: En büyük üstte
  Min Heap: En küçük üstte
- **Graph** → ağ bağlantıları, şehir-yol bağlantıları gibi kısıda

Big-O Notation

Bir algoritmanın büyüme hızıdır

Zaman Karmaşıklığı:

* O(1) → Sabit zaman
  ✓ en iyisidir
  `int x = arr[5];` → 10 elemanlı da 1000 elemanlı da olsa aynı
  Algoritma doğrusaldır ve loop veya recursion içermiyor demektir.

* O(log n) → her adımda işin yarısını atar.
  Binary Search → 1 milyon elemandan biri 20 adımda bulursun
  Kontrol değişkeni her döngüde çarpılır veya bölünür.
  `for (i=1; i<=1000; i*=2)` yada da `for (i=1; i<=1000; i/=2)`

* O(n) → Lineer → Kaç eleman varsa o kadar
  `for (i=0; i<n; i++)`
  - Listeyi baştan sona gezmek
  - Linked list traversal

* O(n log n) → Sağır iyi sıralama algoritması böyledir

Merge sort, Quick sort — iç içe döngüler. Dıştaki toplarken içteki döngü çarpar

```
for(i=1; i<=10; i++)
  for(j=1; j<=10; j*=2)
```

* O(n²) → Yavaş — iki döngü de toplamalı. n×n=n² • Bubble sort, Selection Sort

```
for(i=0; i<10; i++)
  for(j=0; j<10; j++)
```

10 bin eleman olsa bitsin mesela...

* O(n(n+1)/2) — iç döngü dış döngüye bağlı

```
for(i=0; i<10; i++)
  for(j=0; j<i; j++)
```

* Ω(2ⁿ) veya O(n!) → DAAMN RUN GIRL

Kod / İşlem	Big-O
`x = arr[3];`	O(1)
`head = newNode;`	O(1)
Listeyi gezmek	O(n)
Binary search	O(log n)
Bubble sort	O(n²)
Merge sort	O(n log n)
Stack push/pop	O(1)

🧠 LINKED LIST & BIG-O (çok önemli)

Tek yönlü linked list

İşlem	Big-O
Baştan ekleme	O(1)
Sona ekleme (tail yok)	O(n)
Sona ekleme (tail var)	O(1)
Arama	O(n)
i. elemana erişim	O(n)

🌳 AĞAÇLAR & BIG-O

Binary Search Tree (dengeli)

İşlem	Big-O
Arama	O(log n)
Ekleme	O(log n)
Silme	O(log n)

❌ Dengesizse → O(n)

🗃️ ARRAY vs LINKED LIST

İşlem	Array	Linked List
arr[i]	O(1)	❌
Araya ekleme	O(n)	O(1)*
Baştan silme	O(n)	O(1)
Arama	O(n)	O(n)

* konumu biliyorsan

Küçükten büyüğe Sıralama:

$\log n < n < n\log n < n^2 < n^3 < \dots < 2^n < n!$

Queue → FIFO

Stack → LIFO

Stackler

Push → Yığına bir eleman ekler

Pop → Yığının en üstteki elemanını çıkarır

Top → Yığının en üstündeki elemanı gösterir → bu int bi değişkendir

> hepsi void fonksiyonlardır

★ Neden Top kullanıyoruz? Her push/pop'ta tüm diziyi gezmeden bulmak için.
Yani aslında O(1) zamanda işlem yapıyoruz.

Stack tasarım mantığından dolayı Top = -1 olur başta.

Top	Anlamı
-1	Stack boş
0	1 eleman var → çünkü diziler 0'dan başlıyo
1	2 eleman var
⋮	⋮

YA DA global bir stack tanımlamışlardır main'in dışında.

★ Global ve static değişkenler otomatik "0" dır ilk tanımlandığında.

s.top = 0
s.deger[1,2....5] = 0 > olur yani direk

Matematiksel ifadelerin farklı gösterim biçimleri

Infix → Operatörler, operandların arasında bulunur. 2+5*3 gibi

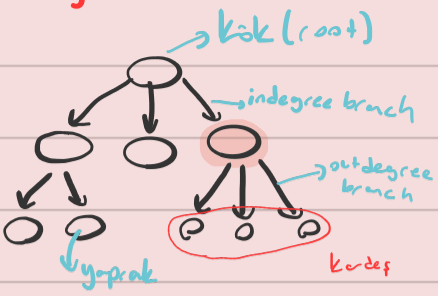
Postfix → Operandların sonunda bulunurlar. 2 3 5 * + → 2+(3*5)

Parantezlerin gerekliliğini kaldırır.

Bilgisayar programlama ve hesaplama cihazlarında kullanılır. Özellikle stacklerde

Prefix → Operatörler önde. + 2 * 3 5 → 2+(3*5)

Ağaçlar



Ata: Kökten düğüme giden yoldaki herhangi bir düğümdür.

Soy: Ebeveyn düğümün altındaki yoldaki herhangi bir düğüm

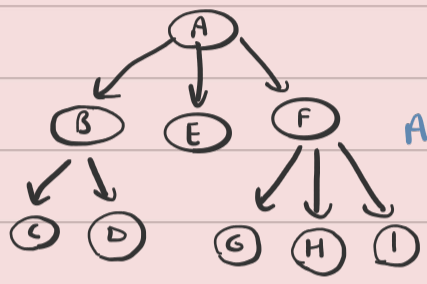
Bir düğümün derinliği (level): Kökten o düğüme kadarki kenarların sayısı

// yüksekliği: Düğümden en derin yaprağa kadar olan // //.

Ağacın boyu: max level of Leaf +1

Tanım gereği, boş ağacın boyu -1 olur.

Parenthetical Listing



A(B(CD)EF(GHI))

her açma parantezi yeni bir seviye başlayışını ifade eder.
her kapama parantezi geçerli seviyeyi tamamlar.

Binary Ağaçlar

İkiden fazla soya sahip olamazlar. 0, 1 veya 2 olabilir. 

! Simetrik olmak zorunda değil. Null da olabilir. (A) da olabilir  da.

N adet düğüm olduğunda, bunları bir binary tree yapsak:

Maksimum yükseklik → $H_{max} = N$ 

Minimum yükseklik → $H_{min} = [\log_2 N] + 1$ 

Ağaç yüksekliği biliniyorsa:

Maksimum Node sayısı → $N_{max} = 2^H - 1$

Minimum → $N_{min} = H$

Balance Factor (ağacın dengesi)

Ağacın, sol ve sağ alt ağaçlarının arasındaki yükseklik farkıdır.

$$\beta = H_L - H_R$$

0, 1 veya -1 ise ağaç dengededir

Complete Tree: Maksimum düğüm sayısına sahipler yaprak kısımlarında 

Nearly Complete Tree: Son seviyedeki tam düğümler solda.  → sıkıntı level yaratır

Traversal

Depth-first traversal: İkinci çocuk işlenmeden önce bir çocuğun bütün soyları işlenir.

Breadth-first traversal: Önce bir seviye işlenip sonra diğer seviyeye geçilir.

Binary Tree Traversal: Ağacın her bir düğümü sadece 1 kere işlenir.

a) Preorder Traversal: Anne/kök, kendini önceye koyuyor.





b) Inorder =: Kendini ortaya alıyor

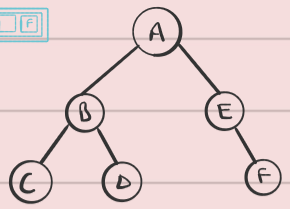




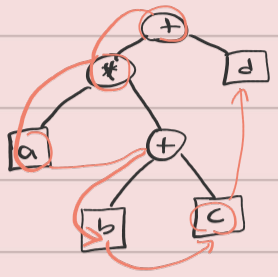
c) Postorder =: Kendini sona alıyor.





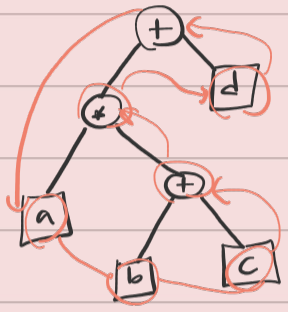


Prefix → önce soldaki çocuğuna bakarım. Sonra onun soldaki kalkıp bittiğde sağ taki çocuğuna bakarım.



+ * a + b c d

Postfix → en derinlerdekine bakıyo bu oe. Operatörleri en son yazıyor. En sona da kendisi



a b c + * d +

infix → bildiğimiz şey yani. ((a * (b + c)) + d)

Huffman Code: Karakterler görülme sıklığına göre dizilir

Binary Search Tree

Sol alt ağaçtaki tüm elemanlar kökten daha küçüktür.

Sağ alt ağaçtaki elemanlar kökten büyük veya köke eşittir.

4 temel BST işlemi vardır:

Traversals: 3'ü Preorder, Inorder, postorder olanlardan
↳ sıralı bir liste döndürür 1,2,3,4,5 gibi

Right-Node-Left traversal → Inorder gibi. ama önce sol yerine hep sağa bakıyor. AZALAN sıralı liste oluşur. 5,4,3,2,1 gibi

Searches

Insertion

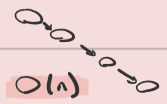
Deletion: Çocuğu yoksa direk silersin.

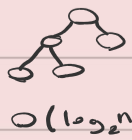
1 çocuğu varsa, elemanı silip çocuğu onun ebeveynine bağlarsın.

2 çocuğu varsa, mesela kök ise, ağaç çok dengesiz olacaktır. Bu yüzden ya sol alt ağacın en büyük elemanını ya da sağ alt ağacın en küçük elemanını köke taşırız.

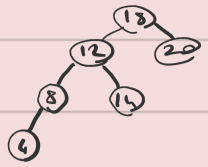
AVL Ağaçları

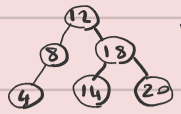
Dengeli: binary ağaçtır. Yani alt ağaçların yükseklik farkı max 1'dir





Left of Left → sağa döndürerek dengelenir

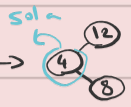




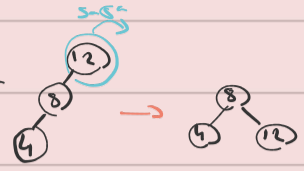
18 sol yaprağın kaybettiğinden 14 soluna geçer

Right of Right → aynı şey ama sola döndür

Right of Left →



Ağacı dengelemek için ilk: doğru no döndürmemiz gerekiyor



Left of Right → Tersi işte

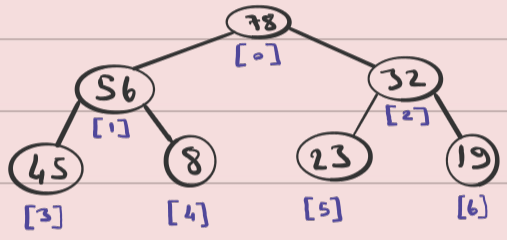


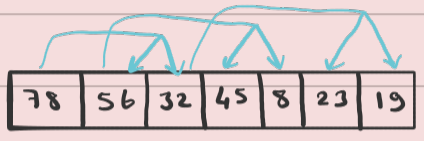
Heap

En büyük değişken köktedir. Daha küçük değişkenler ağacın sol veya sağ alt ağacında

Bağlantılı bir liste yerine bir dizi ile gerçekleştirilir.

Nearly complete veya full complete olacak.  değildir mesela





Array gösterimi

i indeksli bir değişken sol çocuğu → 2i+1

sağ çocuğu → 2i+2' dir.

parent'ı → $\lfloor (i-1)/2 \rfloor$ konumundadır.

Ekleme ve silme için 2 temel algoritmaya ihtiyaç vardır:

reheap up: yeni elemanı sona ekle, parent ile karşılaştır, child < parent ise swap (MIN HEAPTE)

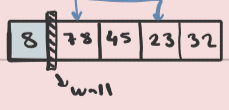
reheap down: Bir eleman silindiğinde, en son eleman root'a taşınır. Sonra çocuklarıyla karşılaştırıp karşılaştırıp aşağı iner.
(en büyük çocuk)

O(log n) çünkü en fazla ağacın yüksekliği kadar yukarı çıkar.

Sort

genelde $O(n^2)$' dir. En iyileri $O(n \log n)$' dir.

straight selection sort →



duvar ilerletiliyor işte sonra $O(n^2)$

heaplerin → n(log n) dir.

Straight insertion → $O(n^2)$ bu direk unsorted kısımdaki elemanı alıp uygun yere arraya konabiliyor

bubble sort → ikişer ikişer karşılaştırıp ilerliyor. Bitince yine, ve yine... $O(n^2)$

quick sort → pivot belirleniyor ona göre. soluna onda küçükler, sağına büyükler. $O(n \log n)$

Queue

Enqueue: Kuyruğun sonuna bir eleman ekliyo

Dequeue: // başından bir eleman siliyor.

Queue Front: Kuyruğun başındaki elemanı okuyor

// Rear: // sonundaki // //

single-server queue: Sadece 1 müşteriye bakıyo (bakkal- bide)

multi-server queue: Aynı anda birden fazla müşteri (banka)

Multiqueue: Birden fazla single-server bir araya gelmiş gibi düşün

Arrival Rate: Müşterilerin servis kuyruğuna gelme sıklığı, süresi
Service Time: Müşterinin isteğini yapmak için gereken süre
Queue Time: Müşterinin kuyrukta bekleme süresi

} Kuyrukların performansını en çok etkileyen faktörler

Finalden Önce Notlar:

Casting: Bir pointer'ın türünü belirtme. int *p; gibi.

Dereference: Bir pointerın işaret ettiği değeri alma. printf("%d", *p);

* Malloc bir fonksiyondur ve bir void türünde adres döndürür. Bir pointera atarsın. int *a = malloc(...
* malloc(5* sizeof(int)); 1 parametre
  calloc(5, sizeof(int)); 2 parametre

Dangling Pointer: free ile heapteki alan serbest bırakıldıktan sonra pointer boşta kalır. p=NULL; yapılmalı.

* int (*fp)(int,int); → int bir değer döndüren bi fonksiyonun pointerıdır. Fonksiyon işaretçisi
  printf("%d", fp(5,6)); → *fp(5,6) da dur, fark etmez.
* int* sayi = (int*) malloc(sizeof(int));
  *sayi = 57; → malloc'ta yer açmadan bunu yapmayın. Yoksa dangling/garbage pointer olur.